FLIGHT SOFTWARE V1 — NOVEMBER BALLOON LAUNCH PLAN

Operating the Base: A Real-Hardware Balloon Flight on the Same Architecture as the Satellite

> *"Sufficiently operate the base of the satellite" — every board commanded and reporting telemetry, reliably, on real hardware, in a real flight. Nothing here needs to be perfect; everything here needs to work.*

Status: companion to `roadmap.md` (revision 5), not a replacement — a disciplined subset of the same plan, scoped to a hard November date

Scope: full bus validation — OBC, Comms, and all four subsystem boards (ADCS, EPS, Thermals, Camera) on real SatNOGS-COMMS hardware. No FPGA compression, no Akida1500, no mission payload.

Companion documents: `docs/roadmap.md`, `docs/roadmap.pdf`, `docs/system_report.pdf`

§ Contents

§ Launch Readiness Scope

"Sufficiently operate the base of the satellite" means: **every board can be commanded and reports telemetry, reliably, on real hardware, in a real flight** — not that every subsystem does its real job. Nothing below needs to be perfect; everything below needs to work.

Must-have — blocks launch MUST

Item	Why it's must-have
OBC (real SatNOGS-COMMS board, Linux) as the CSP hub	Nothing else runs without it
Comms (real STM32H743, Zephyr + libsatnogs-comms) with **working real RF downlink**	Unlike the satellite roadmap, this can't be deferred — ground needs live telemetry to track the balloon during flight
I2C bus, real hardware, connecting OBC + Comms + all four subsystem boards	The actual thing being validated
ADCS, EPS, Thermals, Camera: full CSP command/telemetry scaffolding (Phase 2 DoD) on real hardware	This *is* "operating the base"
Real GPS (SPI, OBC-hosted)	Non-negotiable for a balloon flight — this is how you find the payload afterward
Real EPS battery/power telemetry	Minimum flight-safety visibility — you want to know if power is failing before you lose the link entirely
OBC Services: CI (uplink) + TO (downlink) only	Enough to command and monitor the flight
Pre-flight software readiness checklist (Phase B6)	Catches integration problems on the ground, not in the air

Should-have — do if time allows SHOULD

- Real IMU/magnetometer logging on ADCS (data collection only, no control) — useful data for the real ADCS work later, free if there's slack time.
- Real thermistor logging on Thermals (monitoring only, no heater control) — same reasoning.
- TTS (basic time-tag scheduling) — only if CI proves cumbersome for ground ops to use live during rehearsal. Ground can send commands manually for a single short flight.

Explicitly out of scope for November CUT

Cut	Why
FPGA compression (`roadmap.md` Phase 4A)	A short balloon flight's data volume doesn't need it — that's a real-mission bandwidth problem, not a balloon one.
Akida1500 / neuromorphic compute (Phase 4B)	No neuromorphic hardware for this flight, per direction.
Any mission payload, including camera imagery capture	No payload for this flight, per direction. Camera's board still flies for bus validation — it just doesn't capture anything.
Real ADCS control laws (B-dot, sun-pointing, Kalman)	A balloon gondola has no reaction wheels or magnetorquers — there's nothing to control. ADCS's board proves the bus, not attitude control.
Real Thermals heater control	Passive monitoring only, if anything (Should-have list) — no active thermal control hardware to drive.
LC (limit checking)	Ground watches telemetry live during a short flight and reacts manually. Automating limit response isn't worth the time for one flight.
Command reliability upgrades (Fork A / RDP)	Best-effort I2C is fine for a benign, short, line-of-sight-ish flight. Revisit for the real satellite.

§ Why We're Not Cutting Architectural Corners Under This Deadline

The instruction to "prioritize finishing" and the instruction to "build clean and scalable code" aren't in tension — the second one is what makes the first one possible on a 3-month clock. Specifically, don't skip these even under schedule pressure:

- **Keep the mock/real driver split, even for balloon-only code.** It's tempting to write GPS/battery drivers that only work against real hardware "since we're building this for real anyway." Don't — you need the mock *more* under time pressure, not less, because it's what lets four people build and test four subsystems in parallel without four sets of hardware on four desks simultaneously.
- **Keep sabotage-verified tests, even for the boards that "just" scaffold.** A regression that makes ADCS silently stop responding after two commands is exactly the kind of bug this project has already hit once — the test that catches it costs an hour and prevents a diagnosis session during flight-week crunch.
- **Don't hardcode balloon-specific addresses, ports, or timing into application logic.** Anything balloon-specific (radio frequency, telemetry period, GPS fix interval) belongs in config/env, the same way `OBC_POSITION_PERIOD_SEC` already works today — so the exact same binaries and logic carry forward to the real satellite without a rewrite.

- **Don't invent a balloon-only directory convention.** Everything here lives in the same `apps/`, `shared/`, `platform/` tree as the rest of the codebase. If a balloon-specific need doesn't fit the existing convention, that's a signal to extend the convention, not to bypass it.

§ Timeline Reality Check

Roughly **~190 hours** of work across the phases below (breakdown per phase noted in each section). At a realistic part-time pace and split across a small team using the same parallelization pattern as `roadmap.md`, that's workable in a 3-month window — but only if Phase B5 (hardware bring-up) starts **now**, in parallel with early software phases, not after software is "done." PetaLinux/Yocto builds and STM32H743/Zephyr bring-up are the least predictable items in this whole plan; order any hardware that isn't already on hand this week.

> **RECOMMENDED INTERNAL TARGET**
>
> Hardware- and software-complete with **2–3 weeks of buffer** before the actual launch date for Phase B6's rehearsal and any last-minute fixes it turns up. Don't plan to finish Phase B5 the week of launch.

B1 Foundation

Do all of `roadmap.md`'s Phase 1, tasks 1.1–1.22, exactly as written. Nothing about the balloon flight changes the foundation — the I2C bus migration, the SPI sensor interface pattern, the shared command envelope, the scaffolding template, and the test harness are needed identically whether the destination is a balloon or an orbit.

~40 hours

Sub-phase	What it covers	Reference
1A	Directory & bus convention migration (v_bus → comms_bus.h, CAN dropped, I2C only)	roadmap.md 1.1–1.5
1B	I2C SIM backend (replaces the old SPI/v_bus SIM path)	roadmap.md 1.6–1.10
1C	Shared command/telemetry envelope	roadmap.md 1.11–1.12
1D	Peripheral interface pattern (SPI sensors: IMU, magnetometer, thermistor, GPS)	roadmap.md 1.13–1.17
1E	Scaffolding template, test harness, CI	roadmap.md 1.18–1.21
1F	Foundation Definition of Done (dry run)	roadmap.md 1.22

No changes, no cuts — this is the one phase where the balloon and satellite plans are identical line for line.

B2 Subsystem Scaffolding (Full Bus Validation)

Do `roadmap.md`'s Phase 2 canonical checklist for EPS, Thermals, and Camera, and the Comms Zephyr checklist, exactly as written.

~73 hours (18h × 3 CMake subsystems + ~19h Comms)

Track	Tasks	Reference	Est.
EPS	2.1–2.8	roadmap.md Phase 2A	18h
Thermals	2.1–2.8	roadmap.md Phase 2A	18h
Camera	2.1–2.8	roadmap.md Phase 2A	18h
Comms (Zephyr)	2C.1–2C.7	roadmap.md Phase 2B	19h

ADCS IS A SPECIAL CASE: MOSTLY ALREADY DONE REUSE

ADCS already meets Phase 2's Definition of Done from the earlier position-command work — command in, task fan-out, telemetry out, sabotage-verified test, all already built and working. Once Phase B1's I2C migration lands (which touches ADCS's transport, not its logic), ADCS needs only a quick re-verification pass (rerun 2.6's manual multi-process check and 2.7's test against the new I2C backend), not a rebuild. Budget ~3h for this, not the full 18h.

Definition of Done (all four subsystem boards + Comms): identical to roadmap.md's Phase 2 DoD — builds cleanly, one command flows end-to-end, at least one telemetry reply, mock driver in place, sabotage-verified test, correct CSP address/ports. No balloon-specific relaxation of this bar — this is the actual thing this flight exists to prove.

B3 Balloon-Critical Real Subsystem Work

NEW FOR THIS DOCUMENT — not in roadmap.md, because the satellite roadmap defers real GPS/EPS work to Phase 5 with no urgency. For a balloon flight, GPS and power telemetry are the two pieces of "real" subsystem logic that actually matter for flight safety and recovery.

~13.5 hours

GPS (OBC-hosted, SPI, per Fork B)

#	Task	Depends on	Est.	Done when
B3.1	Select/confirm the GPS module; write a short requirements note (fix rate, accuracy needed for recovery, cold-start time)	Phase B1 (1.17)	1.5h	Module chosen, requirements written down
B3.2	Real SPI GPS driver against `shared/interfaces/gps.h` (parse NMEA or UBX, whichever the module speaks)	B3.1	3h	Real fix data read over SPI, sane lat/lon/alt
B3.3	Wire GPS reads into OBC's telemetry aggregation (Phase B4's TO) so position is downlinked every cycle	B3.2, Phase B4 TO	2h	Position is visible in downlinked telemetry
B3.4	Field test: GPS lock time + accuracy check outdoors, away from the lab (buildings/walls degrade GPS badly)	B3.3	2h	Cold-start lock achieved outdoors within an acceptable time, position accuracy checked against a known point

EPS battery/power telemetry (minimum, no rail-switching logic needed)

#	Task	Depends on	Est.	Done when
B3.5	Real battery voltage/current telemetry driver (ADC or SPI read, new `shared/interfaces/battery.h` if one doesn't exist yet)	EPS Phase B2	3h	Real voltage/current values read, sane ranges
B3.6	Wire EPS battery telemetry into the downlink; confirm behavior under a realistic flight-duration + cold-temperature bench test	B3.5	2h	Voltage/current trend is visible and plausible across a multi-hour bench run

B4 OBC Services, Trimmed

Do `roadmap.md`'s Phase 3 tasks 3.1–3.3 (CI) and 3.9–3.13 (TO + Comms relay) exactly as written. Skip 3.4–3.8 (TTS and LC) entirely for November — see Launch Readiness Scope above for why.

~20 hours

Kept	Reference	Est.
CI (Command Ingest): spec, decode, dispatch	roadmap.md 3.1–3.3	7h
TO (Telemetry Output) + OBC↔Comms relay design/implementation/test	roadmap.md 3.9–3.13	13h

Cut for November	Reference	Revisit for
TTS (Time Tag Scheduler)	roadmap.md 3.4–3.6	Real satellite plan, or if ground ops needs it during rehearsal (Should-have)
LC (Limit Checking)	roadmap.md 3.7–3.8	Real satellite plan — autonomous limit response matters far more for an unattended multi-year mission than a supervised multi-hour flight

B5 Hardware Bring-Up, Balloon Scope

Do `roadmap.md`'s Phase 6, minus the FPGA and Akida1500 items (6.4, 6.5 — neither exists for this flight). This is the least predictable phase in this entire plan — start it in parallel with Phase B1/B2 software work, not after.

~27 hours of active work, but expect this phase to span more calendar time than its hour count suggests

#	Task	Reference	Est.
6.1	Vendor STM32 HAL + real cross-compilation toolchain for the four FreeRTOS boards	roadmap.md 6.1	3h
6.2	OBC: PetaLinux/Yocto build setup for the Zynq PS	roadmap.md 6.2	3h (multi-session — start this first)
6.3	Comms: real Zephyr build + flash to STM32H743 dev hardware	roadmap.md 6.3	3h
6.4	CUT FPGA bring-up — no FPGA on this flight	—	—
6.5	CUT Akida1500 bring-up — no Akida1500 on this flight	—	—
6.6	Fill in `platform/real/drivers/comms_i2c.c` against real I2C HAL/Linux driver headers	roadmap.md 6.6	3h

6.7	Per-subsystem bench bring-up: ADCS	roadmap.md 6.7	3h
6.8	Per-subsystem bench bring-up: EPS, Thermals, Camera	roadmap.md 6.8	3h each (9h)
6.9	Full end-to-end system test, real ground contact through Comms' real radio path	roadmap.md 6.9	3h

B6 Flight Readiness & Pre-Launch Checklist

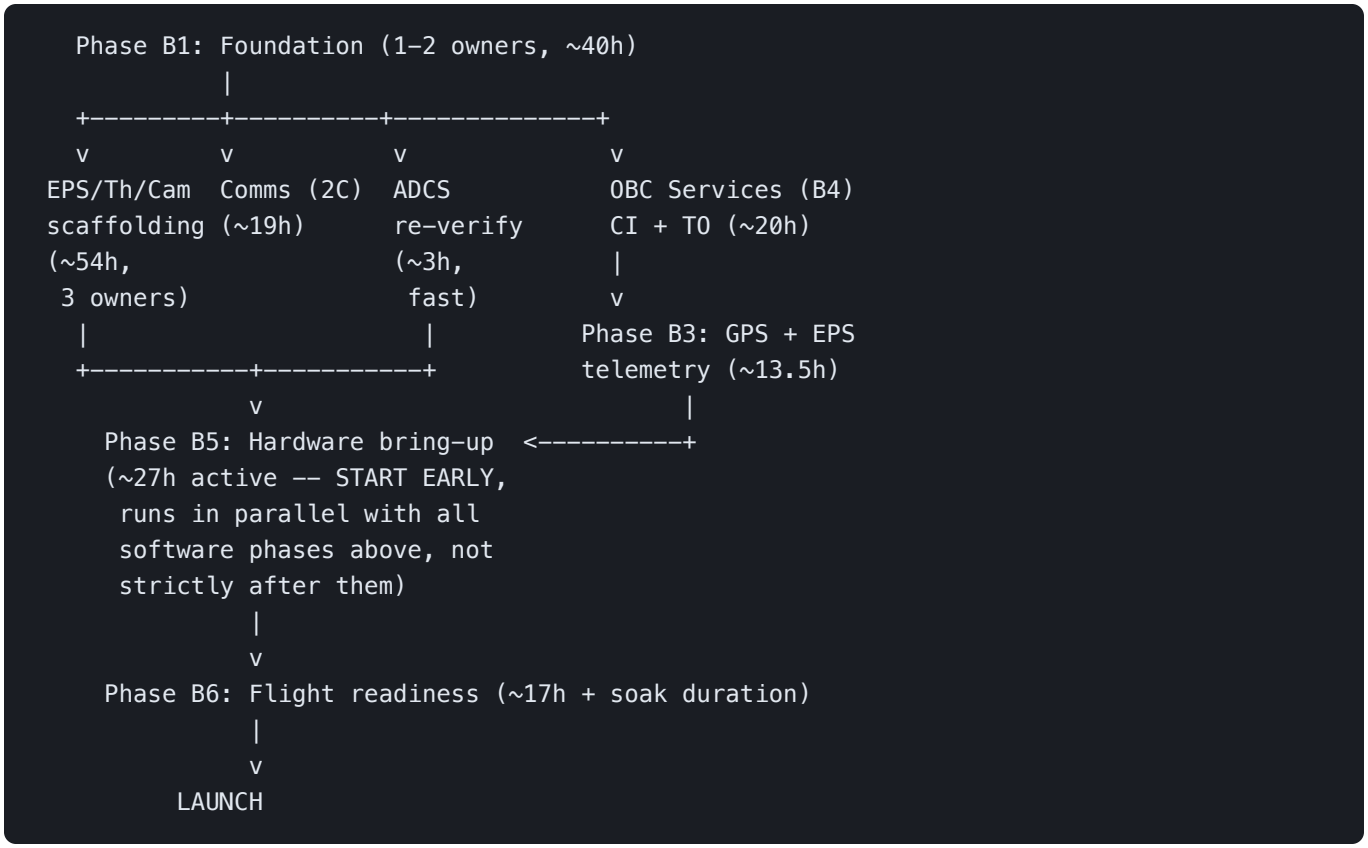
NEW FOR THIS DOCUMENT . These are the software-side checks that catch problems on a bench in October, not in the air in November. Physical balloon/gondola/recovery logistics are out of this document's scope — this is specifically what the flight *software* needs to prove before launch day.

~17 hours of active setup/analysis work, plus the actual soak-test duration running unattended

#	Task	Depends on	Est.	Done when
B6.1	Long-duration soak test: run the full stack (OBC + Comms + all four subsystems) continuously for at least the expected flight duration plus margin, no crash/hang	Phase B2, B4	3h active + soak duration unattended	Full-duration run completes with no crash, no silently-stopped telemetry
B6.2	Ground station range/link test: confirm telemetry downlink holds at realistic distance	Phase B5 (6.9)	3h	Link holds (or degrades gracefully and recovers) at the target test distance
B6.3	Cold-environment check: confirm OBC/Comms/subsystem boards still boot and communicate at expected high-altitude temperatures (fridge/freezer test if no thermal chamber available)	Phase B5	2h	All boards boot and pass a basic command/telemetry check cold
B6.4	Power budget validation: confirm battery capacity covers the full flight duration with margin, using B3.5/B3.6's real telemetry	B3.6	2h	Measured power draw × planned battery capacity clears the flight duration with margin
B6.5	Fault-recovery check: what happens if a subsystem board resets mid-flight — does the rest of the bus keep working, does the reset board rejoin cleanly?	Phase B2	2h	A forced reset of one board doesn't take down the others; the reset board rejoins and resumes reporting
B6.6	Onboard telemetry logging as an RF-loss backup (local log alongside the downlink, e.g. to SD card)	Phase B4	3h	A simulated RF blackout still leaves a complete local telemetry record for post-flight recovery

B6.7	Final go/no-go checklist + full dry run of the exact commands ground ops will send during the real flight	B6.1–B6.6	2h	Dry run executes start-to-finish with no manual workarounds needed

§ Suggested Critical Path & Parallelization



Hardware bring-up (Phase B5) is the one phase that should start **immediately and run alongside everything else**, not wait for software to finish — PetaLinux/Yocto and STM32H743/Zephyr bring-up have the least predictable timelines here. Everything in Phase B2's row can run in parallel across up to four people once Phase B1 lands. Phase B3 needs Phase B4's TO service to have somewhere to send GPS/battery data, but its driver work (B3.2, B3.5) can start as soon as hardware is on hand, independent of B4.

§ Cut-List Quick Reference

What's deliberately not built for November, and where it lives in the full plan instead:

Not built for November	Full satellite plan reference
FPGA compression	roadmap.md Phase 4A
Akida1500 neuromorphic compute	roadmap.md Phase 4B
Real ADCS control laws (B-dot, sun-pointing, Kalman)	roadmap.md Phase 5, ADCS section
Real Thermals heater control	roadmap.md Phase 5, Thermals section
Camera capture/payload logic	roadmap.md Phase 5, Camera PCB section
TTS (Time Tag Scheduler)	roadmap.md Phase 3, tasks 3.4–3.6
LC (Limit Checking)	roadmap.md Phase 3, tasks 3.7–3.8
Command reliability upgrades (RDP)	roadmap.md Fork A

None of this is abandoned — it's simply not on the critical path to a working balloon flight by November. Every one of these picks back up exactly where roadmap.md already scoped it.

Flight Software V1 — November Balloon Launch Plan. Companion to roadmap.md; keep both in sync as decisions change.